

Historique d'activité : Semaine 1 - Bases Python et Flux de Contrôle

Développeur : HMP

I. Configuration de l'environnement

La première étape consiste à installer Python version 3.10 ou ultérieure depuis la distribution officielle python.org. Lors de l'installation, la variable d'environnement PATH est configurée pour permettre l'accès à l'interpréteur Python en ligne de commande depuis n'importe quel répertoire.

Un éditeur de code est sélectionné pour le développement. VS Code avec l'extension Python est retenu comme environnement principal pour son terminal intégré, sa coloration syntaxique et ses capacités de débogage. PyCharm Community Edition constitue une alternative offrant une expérience IDE complète.

L'installation est vérifiée en exécutant 'python --version' dans le terminal. Un fichier test nommé 'hello.py' est créé avec une instruction print. Le fichier est exécuté via 'python hello.py', confirmant que l'environnement est opérationnel.

CONFIGURATION TERMINÉE

- Python 3.10+ installé avec configuration PATH
- VS Code configuré avec l'extension Python
- Premier programme exécuté : print("Hello, World!")

II. Variables et types de données

Les variables Python stockent des données sans déclaration de type explicite. L'interpréteur détermine le type à l'exécution. Les noms de variables commencent par une lettre ou un tiret bas, contiennent des caractères alphanumériques, et sont sensibles à la casse. La convention snake_case est le standard Python.

Cinq types de données principaux sont abordés : les entiers (int) pour les nombres entiers, les flottants (float) pour les valeurs décimales, les chaînes (str) pour le texte entre guillemets, les booléens (bool) pour True/False, et None pour l'absence de valeur. La fonction type() retourne la classe d'un objet.

Les opérations sur les chaînes comprennent la concaténation (+), la répétition (*), et la mesure de longueur (len()). Les chaînes multilignes utilisent les triples guillemets. Les tirets bas améliorent la lisibilité des nombres (1_000_000).

TYPES DE DONNÉES COUVERTS

- int : nombres entiers (42, -10, 1_000_000)
- float : nombres décimaux (19.99, 3.14159, 2.5e6)
- str : texte avec guillemets simples, doubles ou triples
- bool : valeurs True ou False / None : absence de valeur

III. Entrées/sorties et opérateurs

La fonction `print()` affiche des données dans la console. Elle accepte plusieurs arguments et deux paramètres optionnels : `'sep'` définit le séparateur entre les valeurs (espace par défaut), `'end'` définit le terminateur de ligne (retour à la ligne par défaut). Les f-strings permettent un affichage formaté : `f"texte {variable}"`.

La fonction `input()` récupère la saisie utilisateur et retourne une chaîne. Les fonctions `int()` et `float()` convertissent les entrées textuelles en types numériques. La validation est nécessaire car `input()` retourne toujours une chaîne.

Trois catégories d'opérateurs sont couvertes. Arithmétiques : `+`, `-`, `*`, `/`, `//` (division entière), `%` (modulo), `**` (puissance). Comparaison : `==`, `!=`, `<`, `>`, `<=`, `>=` (retournent des booléens). Logiques : `and`, `or`, `not` (combinent des expressions booléennes).

RÉFÉRENCE DES OPÉRATEURS

- Arithmétiques : `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Comparaison : `==`, `!=`, `<`, `>`, `<=`, `>=`
- Logiques : `and`, `or`, `not`

IV. Structures de contrôle et projet appliqué

Les instructions conditionnelles contrôlent l'exécution selon des expressions booléennes. L'instruction `if` exécute un bloc lorsque la condition est `True`. La clause `else` fournit une alternative quand la condition est `False`. Le mot-clé `elif` permet d'enchaîner plusieurs conditions.

Python utilise l'indentation pour définir les blocs de code. Une indentation cohérente de 4 espaces par niveau est requise. Les conditions sont combinées avec les opérateurs logiques (`and`, `or`) et peuvent être imbriquées.

Le projet de fin de semaine est un classificateur de notes. Le programme reçoit un score numérique via `input()`, le convertit en entier, puis utilise des instructions `if/elif/else` enchaînées pour attribuer une lettre (A, B, C, D ou F) selon des seuils définis. Le résultat est affiché avec une f-string montrant le score et la note correspondante.

IMPLÉMENTATION DU PROJET

- `input()` reçoit le score numérique en chaîne
- `int()` convertit la chaîne en valeur entière
- `if/elif/else` évalue les seuils et attribue la note
- f-string affiche le score et la lettre de note